

ЛАБОРАТОРНА РОБОТА № 1 ШАБЛони ПОВЕДІНКИ.

1.1 Мета роботи

Вивчити шаблони поведінки та отримати базові навички із застосування шаблонів Ітератор (Iterator), Посередник (Mediator), Спостерігач (Observer), Стратегія (Strategy), Ланцюг обов'язків (Chain of Responsibility), Відвідувач (Visitor).

1.2 Основні теоретичні відомості

Шаблони поведінки – шаблони проектування, що пов'язані з алгоритмами та розподілом обов'язків поміж об'єктів. Мова в них йде не тільки про самі об'єкти та класи, але й про типові способи їхньої взаємодії. Шаблони поведінки характеризують складний потік керування, котрий досить важко прослідкувати під час виконання програми. Увага акцентована не на потоці керування, а на зв'язках між об'єктами. У шаблонах поведінки рівня класу використовується спадкування щоб розподілити поведінку поміж різними класами. У шаблонах поведінки рівня об'єкта використовується композиція. Деякі з них описують, як за допомогою кооперації багато рівноправних об'єктів пораються із завданням, котре жодному з них не під силу. Тут є важливим те, як об'єкти отримують інформацію про існування один одного. Об'єкти-колеги можуть зберігати посилання один на одного, але це посилює ступінь зв'язаності системи. За максимального рівня зв'язаності кожному об'єкту довелося би мати інформацію про всі інші. Деякі з шаблонів поведінки вирішують цю проблему.

1.2.1 Шаблон проектування Ітератор (Iterator)

Призначення. Надає спосіб послідовного доступу до всіх елементів складеного об'єкта, не розкриваючи його внутрішнього улаштування.

Складений об'єкт, скажімо список, дерево, хеш-таблиця або масив, повинен надавати спосіб доступу до своїх елементів, не розкриваючи їхню внутрішню структуру. Більш того, іноді треба обходити список по-різному, у залежності від задачі, що вирішується. При цьому немає ніякого бажання засмічувати інтерфейс класу Список усілякими операціями для усіх потрібних варіантів обходу,

навіть якщо їх усі можна передбачити заздалегідь. Крім того, іноді треба, щоб в один момент часу існувало декілька активних операцій обходу списку.

Все це призводить до необхідності реалізації шаблону Ітератор. Його основна ідея у тому, щоб за доступ до елементів та обхід списку відповідав не сам список, а окремий об'єкт-ітератор.

У класі Ітератор означений інтерфейс для доступу до елементів списку. Об'єкт цього класу прослідковує поточний елемент, тобто він володіє інформацією, які з елементів вже відвідувались.

Також відомий під назвою курсор (Cursor).

Застосування. Використовується:

- для доступу до змісту агрегованих об'єктів не розкриваючи їхнє внутрішнє улаштування;
- для підтримки декількох активних обходів одного й того ж агрегованого об'єкта;
- для подання уніфікованого інтерфейсу з метою обходу різноманітних агрегованих структур (тобто для підтримки поліморфної ітерації).

Структура шаблону наведена на рис. 3.1.

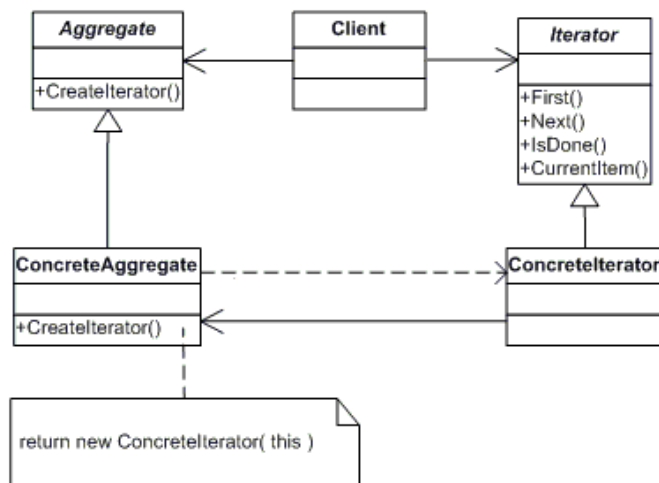


Рисунок 1.1 – Структура шаблону Iterator

Учасниками шаблону є:

- **Iterator** – ітератор:

- визначає інтерфейс для доступу та обходу елементів
- **ConcreteIterator:**
 - реалізує інтерфейс класу Iterator;
 - слідує за поточною позицією під час обходу агрегату;
- **Aggregate:**
 - визначає інтерфейс для створення об'єкта-ітератора;
- **ConcreteAggregate :**
 - реалізує інтерфейс створення ітератора та повертає екземпляр відповідного класу ConcreteIterator

Відносини. *ConcreteIterator* відслідковує поточний об'єкт у агрегаті та може вирахувати наступний.

Результати. Шаблон підтримує різні способи перебору агрегату, одночасно можуть бути активні декілька переборів.

1.2.2 Шаблон проектування Посередник (Mediator)

Призначення. Визначає об'єкт, що інкапсулює спосіб взаємодії множини об'єктів. Посередник забезпечує слабку зв'язаність системи, звільняючи об'єкти від необхідності явно посилатися один на одного, і дозволяючи тим самим незалежно змінювати взаємодії між ними.

Застосування. Використовується коли:

- існують об'єкти, зв'язки між котрими досить складні та чітко задані. Отримані при цьому залежності не структуровані та важкі для розуміння;
- не можна повторно використовувати об'єкт, оскільки він обмінюється інформацією з багатьма іншими об'єктами;
- поведінка, розподілена між кількома класами, повинна піддаватися налагодженню без створювання множини підкласів.

Структура шаблону наведена на рис. 3.2.

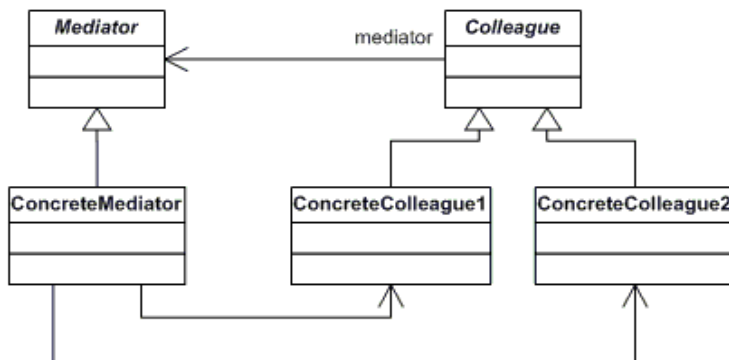


Рисунок 1.2 – Структура шаблону Mediator

Учасниками шаблону є:

- **Mediator** – посередник:
 - визначає інтерфейс для обміну інформацією з об'єктами Colleague;
- **ConcreteMediator** – конкретний посередник:
 - реалізує кооперативну поведінку, координуючи дії об'єктів Colleague;
 - володіє інформацією про колег, та підраховує їх;
- Класи **Colleague** – колеги:
 - кожному класу Colleague відомо про свій об'єкт Mediator;
 - усі колеги обмінюються інформацією виключно через посередника, інакше за його відсутності їм довелося б спілкуватися між собою напряму.

Відносини. Колеги посилають запити посередникові та отримують запити від нього. *Посередник* реалізує кооперативну поведінку шляхом переадресації кожного запиту відповідному колезі (або декільком з них).

Результат. Шаблон знижує кількість класів, що породжуються, усуває зв'язаність між колегами, спрощує протоколи взаємодії об'єктів, абстрагує спосіб кооперації об'єктів та централізує управління.

1.2.3 Шаблон проектування Спостерігач (Observer)

Призначення. Визначає залежність типу “один до багатьох” між об'єктами таким чином, що при зміні стану одного об'єкта всі

залежних від нього сповіщаються про цю подію.

Також відомий як “підлегли” (Dependents) або “видавець-передплатник” (Publisher-Subscriber).

Застосування. Використовується коли:

- існує, як мінімум, один об'єкт, що розсилає повідомлення і є не менше одного одержувача повідомлень, причому їхня кількість і склад можуть змінюватися під час роботи програми;
- коли при модифікації одного об'єкту необхідно змінити інші і не відомо, скільки таких об'єктів необхідно змінити;
- коли один об'єкт повинен сповіщати інших, не роблячи припущень про об'єкти, що сповіщаються, іншими словами потрібно, щоб об'єкти не були тісно зв'язані між собою.

Цей шаблон часто застосовують в ситуаціях, в яких відправника повідомлень не цікавить, що роблять одержувачі з наданою їм інформацією.

Структура шаблону наведена на рис. 3.3.

Учасниками шаблону є:

- **Subject** – суб'єкт:
 - має інформацію про своїх спостерігачів, за суб'єктом можуть «слідкувати» будь-яка кількість спостерігачів;
 - надає інтерфейс, що визначає методи для додавання, видалення та оповіщення спостерігачів;
- **Observer** – спостерігач:
 - визначає інтерфейс для оновлення об'єктів, які повинні бути сповіщені про зміну суб'єкту;
- **ConcreteSubject** – конкретний суб'єкт:
 - зберігає стан, що становить інтерес для конкретного спостерігача *ConcreteObserver*;
 - надсилає інформацію своїм спостерігачам, коли відбувається зміна;
- **ConcreteObserver** – конкретний спостерігач:
 - зберігає посилання на об'єкт класу *ConcreteSubject*;
 - зберігає дані, які повинні бути узгоджені з даними суб'єкту;
 - реалізує інтерфейс оновлення, визначений в класі *Observer*, для підтримки узгодженості з суб'єктом.

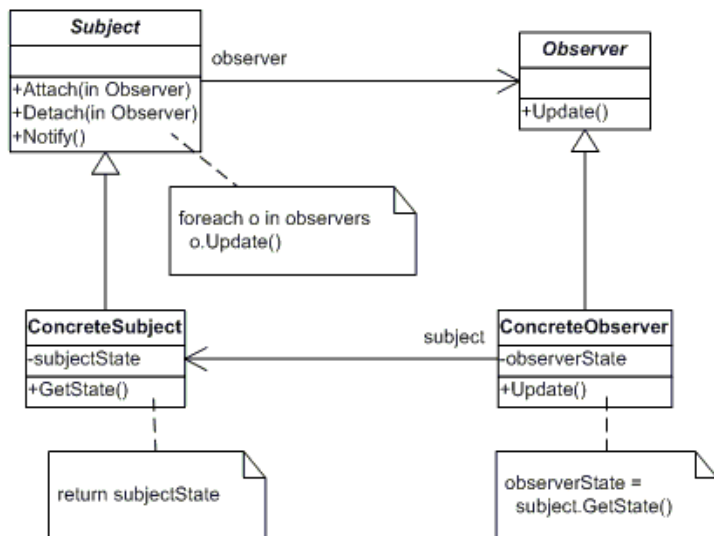


Рисунок 1.3 – Структура шаблону Observer

При зміні спостережуваного об'єкту, оповіщення спостерігачів може бути реалізоване за такими сценаріями:

- спостережуваний об'єкт надсилає, кожному із зареєстрованих спостерігачів, всю потенційно релевантну інформацію (примусове розповсюдження);
- спостережуваний об'єкт надсилає, кожному із зареєстрованих спостерігачів, лише повідомлення про те що інформація була змінена, а кожен із спостерігачів, за необхідності, самостійно здійснює запит необхідної інформації у спостережуваного об'єкта (розповсюдження за запитом).

Відносини. Об'єкт *ConcreteSubject* оповіщає своїх спостерігачів про будь-яку зміну, що могла би привести до неузгодженості станів спостерігача та суб'єкта.

Отримавши від конкретного суб'єкта сповіщення про зміну, об'єкт *ConcreteObserver* може запитати у суб'єкта додаткову інформацію, яку використає для того, щоб опинитися в стані, узгодженому зі станом суб'єкту.

Об'єкт *Observer*, що ініціює запит на зміну, відкладає своє оновлення до отримання сповіщення від суб'єкту. Операція *Notify* не завжди викликається суб'єктом. Її може викликати і спостерігач і

сторонній об'єкт.

Результат. Шаблон дозволяє змінювати суб'єкти та спостерігачі незалежно один від одного. Суб'єкти дозволяється повторно використовувати без участі спостерігачів та навпаки. Це надає можливість додавати нових спостерігачів без модифікації суб'єкта чи інших спостерігачів.

1.2.4 Шаблон проектування Стратегія (Strategy)

Призначення. Суть шаблону полягає у тому, щоб створити декілька схем поведінки для одного об'єкту та винести в окремий клас.

Відомий ще під назвою політика (Policy).

Існують системи, поведінка яких визначається відповідно до певного роду алгоритмів. Всі вони подібні між собою: призначені для вирішення спільних задач, мають однаковий інтерфейс для користування, але відрізняються тільки "поведінкою", тобто реалізацією. Користувач, налаштувавши програму на потрібний алгоритм, отримує потрібний результат.

Застосування. Використовується коли:

- є багато споріднених класів, що відрізняються лише поведінкою (стратегія дозволяє зконфігурувати клас, задавши одну з можливих поведінок);
- необхідно мати декілька варіантів алгоритму (наприклад, можна визначити два варіанти алгоритму, один з яких потребує більше часу, а інший – більше пам'яті; стратегії дозволяється використовувати, коли варіанти алгоритмів реалізовані у вигляді ієрархій класів);
- в алгоритмі містяться складні специфічні дані, про які клієнт не повинний знати;
- в класі визначено багато поведінок, що представлені у вигляді розгалужених умовних операторів.

Структура шаблону наведена на рис. 3.4.

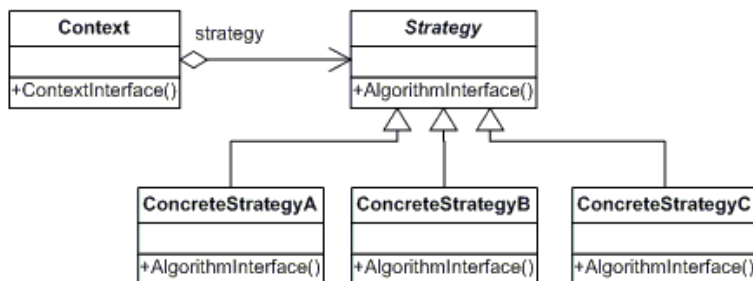


Рисунок 1.4 – Структура шаблону Strategy

Учасниками шаблону є:

- **Strategy** – стратегія:
 - визначає загальний інтерфейс для всіх алгоритмів, що підтримуються. Клас *Context* користується цим інтерфейсом для виклику конкретного алгоритму, визначеного в класі *ConcreteStrategy*;
- **ConcreteStrategy** – конкретна стратегія:
 - реалізує алгоритм, що використовує інтерфейс, визначений в класі *Strategy*;
- **Context** – контекст:
 - конфігурується об'єктом класу *ConcreteStrategy*;
 - зберігає посилання на об'єкт класу *Strategy*;
 - може визначати інтерфейс, що дозволяє об'єкту *Strategy* отримати доступ до даних контексту.

Створюється декілька класів *ConcreteStrategy*, кожен з яких містить один і той же поліморфний метод *AlgorithmInterface*. Об'єкт стратегії зв'язується з контекстним об'єктом (тим об'єктом, до якого застосовується алгоритм).

Відносини. Класи *Strategy* та *Context* взаємодіють для реалізації обраного алгоритму. Контекст може передати до стратегії всі необхідні алгоритму дані під час його виклику. Замість цього контекст може дозволити звертатися до своїх операцій в необхідні моменти, передавши посилання на самого себе операціям класу *Strategy*.

Контекст переадресує запити своїх клієнтів до об'єкту-стратегії. Зазвичай клієнт створює об'єкт *ConcreteStrategy* та передає його контексту, після чого клієнт «спілкується» виключно з контекстом. Часто в розпорядженні клієнта знаходиться декілька класів

ConcreteStrategy, які він може обирати.

І Стратегія, і Декоратор можуть використовуватися для зміни поведінки конкретних класів. Переваги Стратегії полягають в тому, що інтерфейс кастомізації не співпадає з публічним інтерфейсом та може бути більш зручним, а недолік в тому, що необхідно з самого початку проектувати клас з можливістю реєстрації стратегій.

1.2.5 Шаблон проектування Ланцюг обов'язків (Chain of Responsibility)

Призначений для організації в системі рівнів відповідальності, коли запит повинен бути оброблений декількома об'єктами.

Застосування. Використовується коли:

- є більш ніж один об'єкт, здатний обробити запит, при цьому реальний обробник заздалегідь не відомий і повинен бути знайдений автоматично;
- необхідно відправити запит одному з декількох об'єктів, не вказуючи явно одержувача;
- набір об'єктів, що можуть обробити запит повинен задаватися динамічно під час виконання програми.

Структура шаблону наведена на рис. 3.5.

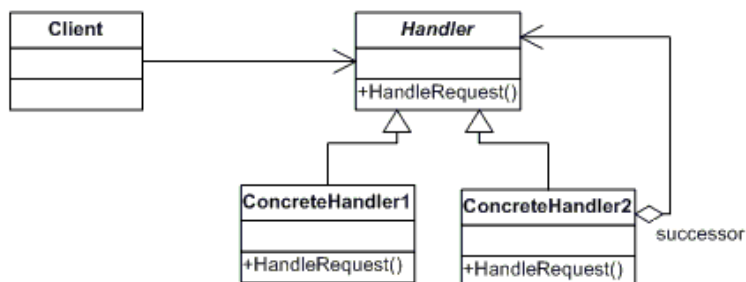


Рисунок 1.5 – Структура шаблону Chain of Responsibility

Учасниками шаблону є:

- **Handler** – обробник:
 - визначає інтерфейс для обробки запитів;
 - (необов'язково) реалізує зв'язок з приймачем;
- **ConcreteHandler** – конкретний обробник:
 - обробляє запит, за який відповідає;
 - має доступ до свого приймача;

- якщо *ConcreteHandler* здатен обробити запит, то так і робить, якщо ні, то відправляє його своєму приймачу;
- **Client** – клієнт:
 - відправляє запит деякому об'єкту *ConcreteHandler* в ціпочці.

Відносини. Коли клієнт ініціює запит, він просувається по ціпочці доти, поки деякий об'єкт *ConcreteHandler* не візьме на себе відповідальність за його обробку.

Результат. Шаблон послаблює зв'язаність (об'єкт не зобов'язаний "знати" про існування всіх інших об'єктів та про те, хто саме опрацює його запит) та дозволяє підвищити гнучкість при розподілі обов'язків між об'єктами. Але немає гарантій, що запит буде оброблений, оскільки він не має явного одержувача.

1.2.6 Шаблон проектування Відвідувач (Visitor)

Призначення. дозволяє відділити певний алгоритм від елементів, на яких алгоритм має бути виконаний, таким чином можливо легко додати або ж змінити алгоритм без змін щодо елементів системи. Практичним результатом є можливість додавання нових операцій в існуючі структури об'єкта без зміни цих структур.

Відвідувач дозволяє додавати нові віртуальні функції в родинні класи без зміни самих класів, натомість, один відвідувач створює клас, який реалізує всі відповідні спеціалізації віртуальної функції. Відвідувач бере приклад посилання в якості вхідних даних і реалізується шляхом подвійної диспетчеризації.

Застосування. Використовується коли:

- в структурі наявні об'єкти декількох класів з різними інтерфейсами і необхідно виконувати над ними операції, що залежать від конкретних класів;
- над об'єктами, що входять в состав структури, необхідно виконувати різноманітні, не зв'язані між собою операції та не бажано «ускладнювати» класи цими операціями;
- класи, що установлюють структуру об'єктів, змінюються рідко, але часто додаються нові операції над структурою.

Структура шаблону наведена на рис. 3.6.

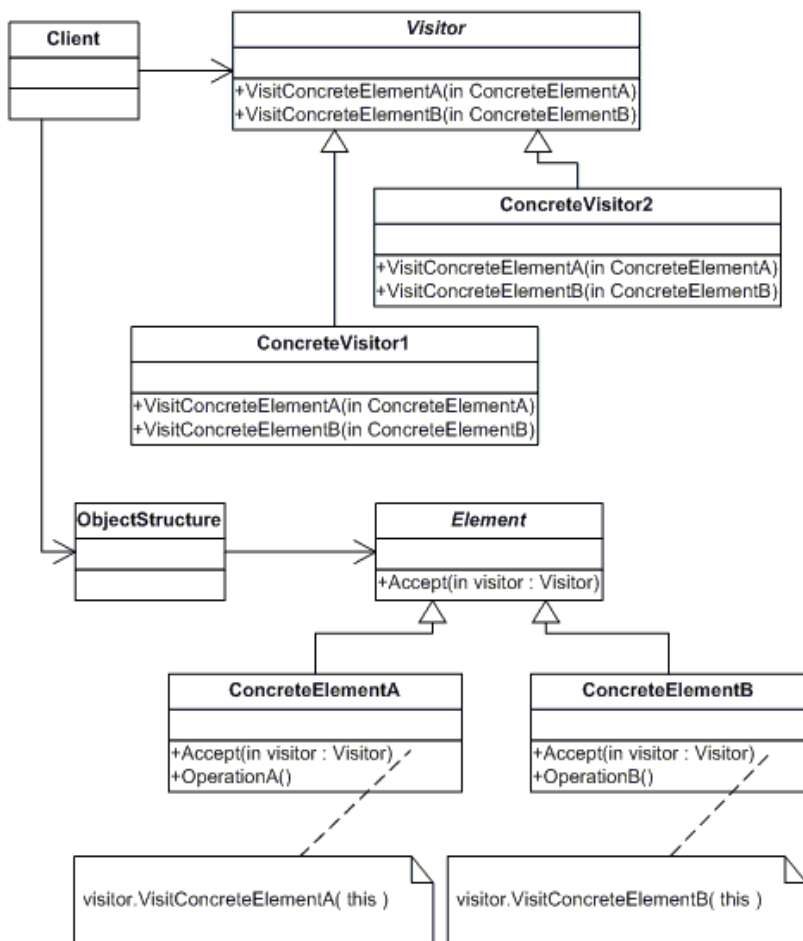


Рисунок 1.6 – Структура шаблону Visitor

Учасниками шаблону є:

– **Visitor** – відвідувач:

- оголошує операцію *Visit* для кожного класу *ConcreteElement* в структурі об'єктів. Ім'я та сигнатура цієї операції ідентифікують клас, який посилає відвідувачу запит *Visit*. Це дозволяє відвідувачу визначити елемент якого конкретного класу він відвідує. Маючи цю інформацію, відвідувач може звертатися к елементу

- безпосередньо через його інтерфейс;
- **ConcreteVisitor**– конкретний відвідувач:
 - реалізує всі операції, оголошені в класі *Visitor*. Кожна операція реалізує фрагмент алгоритму, визначеного для класу відповідного об'єкту в структурі. Клас *ConcreteVisitor* надає контекст для цього алгоритму та зберігає його локальний стан. Часто в цьому стані акумулюються результати, отримані в процесі обходу структури;
- **Element** – елемент:
 - визначає операцію *Accept*, яка приймає відвідувача в якості аргументу
- **ConcreteElement**– конкретний елемент:
 - реалізує операцію *Accept*;
- **ObjectStructure** – структура об'єктів:
 - може перелічувати свої елементи;
 - може надавати відвідувачу високорівневий інтерфейс для відвідування власних елементів;
 - може бути як складеним об'єктом (компонувальник), так і колекцією, наприклад, списком або множиною.

Відносини. Клієнт, який використовує даний шаблон, повинен створити об'єкт класу *ConcreteVisitor*, а потім відвідати кожен елемент структури. *Visitor* оголошує операцію *VisitConcreteElement* для кожного класу *ConcreteElement* (ім'я та сигнатура даної операції ідентифікують клас, елемент якого відвідує *Visitor* – тобто, відвідувач може звертатися до елементу безпосередньо). *ConcreteVisitor* реалізує всі операції, оголошення в класі *Visitor*.

Кожна операція реалізує фрагмент алгоритму, визначеного для класу відповідного об'єкта в структурі. Клас *ConcreteVisitor* надає контекст для цього алгоритму і зберігає його локальний стан. *Element* визначає операцію *Accept*, яка приймає *Visitor* як аргумент, *ConcreteElement* реалізує операцію *Accept*, яка приймає *Visitor* як аргумент. *ObjectStructure* може перерахувати свої аргументи і надати відвідувачеві високорівневий інтерфейс для відвідування своїх елементів.

Логічним є використання цього шаблону, якщо в структурі присутні об'єкти багатьох класів з різними інтерфейсами, і необхідно виконати над ними операції, що залежать від конкретних класів, або

якщо класи, що визначають структуру об'єктів змінюються рідко, але нові операції над цією структурою додаються часто.

Результати. Шаблон спрощує додавання нових операцій, об'єднує споріднені операції в класі *Visitor*. При цьому ускладнюється додавання нових класів *ConcreteElement*, оскільки потрібно оголошення нової абстрактної операції в класі *Visitor*.

Сукупна структура об'єктів елементу може визначатися за допомогою шаблону Компонувальник (Composite). Для обходу може використовуватися Ітератор (Iterator).

Шаблон Відвідувач демонструє класичний прийом відновлення інформації про втрачені типи, не вдаючись до понижуючого приведення типів(динамічне приведення).

1.3 Завдання на лабораторну роботу

1.3.1 Вивчити шаблони поведінки, їхню загальну характеристику та призначення. Для шаблонів згідно варіанту:

- вивчити призначення шаблону, альтернативні назви, призначення, випадки коли його застосування є доцільним та результати такого застосування;
- знати особливості реалізації шаблону, споріднені шаблони, відомі випадки його застосування в програмних додатках;
- вільно володіти структурою шаблону, призначенням його класів та відносинами між ними.

1.3.2 За допомогою джерел зі списку рекомендованих для виконання лабораторної роботи, виконати програмну реалізацію шаблонів поведінки.

№	Шаблони
1	Ітератор, Відвідувач
2	Посередник, Ланцюг обов'язків
3	Спостерігач, Стратегія
4	Стратегія, Ітератор
5	Ланцюг обов'язків, Відвідувач
6	Відвідувач, Стратегія
7	Ітератор, Ланцюг обов'язків
8	Посередник, Стратегія
9	Спостерігач, Ітератор
10	Стратегія, Відвідувач

1.4 Зміст звіту

- 1.4.1 Тема та мета роботи.
- 1.4.2 Короткі теоретичні відомості.
- 1.4.3 Результати роботи.
- 1.4.4 Висновки, що відображують результати виконання роботи та їх критичний аналіз.

1.5 Контрольні запитання

- 1.5.1 Призначення шаблонів поведінки для проектування ПЗ.
- 1.5.2 Коротка характеристика кожного розглянутого шаблону поведінки.
- 1.5.3 Назви, призначення та мотивація шаблону Iterator.
- 1.5.4 Структура шаблону Iterator та його учасники.
- 1.5.5 Особливості реалізації шаблону Iterator. Результат використання шаблону.
- 1.5.6 Назви, призначення та мотивація шаблону Mediator.
- 1.5.7 Структура шаблону Mediator та його учасники.
- 1.5.8 Особливості реалізації шаблону Mediator. Результат використання шаблону.
- 1.5.9 Назви, призначення та мотивація шаблону Observer.
- 1.5.10 Структура шаблону Observer та його учасники.
- 1.5.11 Особливості реалізації шаблону Observer. Результат використання шаблону.
- 1.5.12 Шаблони, які використовуються сумісно з Iterator, Mediator та Observer.
- 1.5.13 Назви, призначення та мотивація шаблону Strategy.
- 1.5.14 Структура шаблону Strategy та його учасники.
- 1.5.15 Особливості реалізації шаблону Strategy. Результат використання шаблону.
- 1.5.16 Назви, призначення та мотивація шаблону Chain of Responsibility.
- 1.5.17 Структура шаблону Chain of Responsibility та його учасники.
- 1.5.18 Особливості реалізації шаблону Chain of Responsibility. Результат використання шаблону.
- 1.5.19 Назви, призначення та мотивація шаблону Visitor.
- 1.5.20 Структура шаблону Visitor та його учасники.

1.5.21 Особливості реалізації шаблону Visitor. Результат використання шаблону.

1.5.22 Шаблони, які використовуються сумісно з Strategy, Chain of Responsibility та Visitor.